

Meta Harness / Project Factory

v1.2 실행형 PRD·기술명세·테스트명세서

빈 폴더에서 시작해 MVP 하네스부터 완성형 Agentic Software Factory 까지 구현하기 위한 최종 준비 문서

문서 포지션

이 문서는 단일 PRD 가 아니라 PRD + SRS + 아키텍처 명세 + 구현 계획 + 테스트 명세를 합친 실행형 명세서다.
목표는 논문이 아니라 실제 구현 가능한 커리어용 DevTool 프로젝트 산출물이다.

항목	내용
문서 버전	v1.2.0 Implementation Ready
작성 기준일	2026-06-14
시작 전제	아무것도 없는 빈 폴더
1 차 목표	MVP: planning scaffold → factory bootstrap → L0 worktree runner → artifact 생성
최종 목표	Repo-resident Agentic Software Factory
주요 산출물	PDF, DOCX, Markdown, 다이어그램 포함

문서 구성

1. Executive Summary
2. PRD - 제품 요구사항 정의
3. Scope - MVP 부터 완성형까지
4. 시스템 아키텍처
5. 기능 요구사항
6. 기술 명세
7. 빈 폴더에서 시작하는 구현 절차
8. 단계별 추가 항목과 기대효과
9. 테스트 명세서
10. 완료 기준과 인수 기준
11. 리스크 및 대응
12. 포트폴리오 활용 전략
13. Appendix A. 계약 파일 예시
14. Appendix B. 다이어그램 카탈로그

1. Executive Summary

Meta Harness / Project Factory 는 AI coding agent 가 실제 프로젝트에서 안전하게 일할 수 있도록, 기획 산출물·작업 계약·실행 환경·검증·산출물 회수·업그레이드 루프를 하나의 repo-resident 개발 공장으로 구성하는 DevTool 프로젝트다. Meta Harness 는 특정 프로젝트를 소유하지 않고, Target Project Repo 안에 프로젝트별 개발 공장을 생성한다. 이후 Target Repo 는 자체적으로 작업공간을 만들고 Agent Worker 를 실행하며 patch/report/PR 같은 내구성 있는 산출물을 남긴다.

핵심 문장

Meta 가 기획 프로세스를 실행한다.
Target Repo 가 기획 산출물을 소유한다.
Meta 가 그 산출물을 읽고 개발 공장을 생성한다.
생성된 개발 공장은 Target Repo 안에 존재한다.
이후 Target Repo 는 자체적으로 작업공간을 만들고 Agent Worker 를 실행하고 PR 을 생산할 수 있다.

구분	정의	핵심 책임
Meta Harness Platform	공장을 만드는 공장	템플릿, 생성기, 정책, 스키마, 평가, 일반화된 학습 관리
Target Project Repo	프로젝트별 개발 공장	기획, 코드, 인프라, CI/CD, 실행 산출물의 source of truth
Ephemeral Workcell	작업별 임시 작업장	worktree/container/kind/action 기반 작업 실행
Agent Worker	일회용 작업자	Task Packet 에 따라 코드 수정, 테스트, artifact 생성
Meta Learning Layer	일반화된 개선 계층	raw data 가 아닌 sanitized signal 만 수집해 템플릿/정책 개선

2. PRD - 제품 요구사항 정의

2.1 제품 한 줄 정의

One-liner

승인된 기획 산출물을 기반으로 Target Repo 안에 AI-assisted 개발 공장을 생성하고, 작업별 임시 Workcell 에서 Agent Worker 를 실행해 patch/report/PR 을 생산하는 repo-resident DevTool.

2.2 문제 정의

- AI coding agent 는 코드를 빠르게 만들 수 있지만, 기획 산출물과 구현 작업 사이의 계약이 약하면 결과가 흔들린다.
- 기존 repo 에 AI agent 를 바로 투입하면 수정 가능 범위, 금지 경로, 테스트 기준, 보안 경계가 불명확해진다.
- agent 실행 결과를 재현하거나 검증하기 어렵고, 실패 경험을 다음 프로젝트에 일반화하는 구조가 부족하다.
- 포트폴리오 관점에서는 단순 앱보다 DevTool·아키텍처·테스트·문서화 역량을 함께 보여줄 수 있는 프로젝트가 필요하다.

2.3 목표

ID	목표	설명
G-01	기획 산출물 소유권 고정	PRD, MVP scope, acceptance criteria, build handoff 를 Target Repo 가 소유한다.
G-02	승인 기반 공장 생성	human-approved Build Handoff 가 없으면 full factory bootstrap 을 막는다.
G-03	작업 계약화	Task Packet 으로 objective, scope, tests, budgets, artifacts 를 명시한다.
G-04	임시 실행 환경	L0 worktree 부터 시작해 L1 container, L2 kind, L3 GitHub Action 으로 확장한다.
G-05	검증 가능한 산출물	patch.diff, run-result.json, summary.md, test-report 등을 표준 artifact 로 남긴다.
G-06	데이터 경계 유지	Meta 는 raw docs/code/logs/secrets 를 저장하지 않고 sanitized signal 만 학습한다.

2.4 Non-goals

- 처음부터 완전 자율 개발 플랫폼을 만들지 않는다.
- 처음부터 Kubernetes, GitHub App, Checks API, Productization auto-fix 를 구현하지 않는다.
- 모든 agent runtime 을 동시에 지원하지 않는다. MVP 는 Shell Adapter 와 Codex Adapter 중 하나로 시작한다.
- Meta Harness 가 프로젝트별 원문 기획서, 소스코드, 로그를 중앙 저장하지 않는다.
- 논문 또는 연구 실적이 1 차 목표가 아니다. 1 차 목표는 작동하는 DevTool 포트폴리오다.

2.5 대상 사용자

사용자	니즈	제품이 제공하는 가치
개인 개발자/포트폴리오 제작자	아이디어를 구조화하고 구현 흐름을 자동화하고 싶다.	기획 → 공장 생성 → 작업 실행 → 산출물 생성 데모를 제공한다.
프론트엔드/풀스택 지망생	단순 앱이 아닌 시스템 설계 역량을 보여주고 싶다.	CLI, 파일 계약, 테스트, UI Dashboard 까지 확장 가능한 포트폴리오가 된다.
소규모 팀/스터디	AI agent 사용 시 범위와 품질 기준을 통제하고 싶다.	Task Packet, forbidden scope, quality gate, run artifact 구조를 제공한다.
미래 확장 사용자	프로젝트별 하네스를 생성·업그레이드하고 싶다.	manifest.lock, state machine, upgrade PR 기반 lifecycle 을 제공한다.

2.6 성공 지표

지표	MVP 기준	완성형 기준
End-to-end demo	빈 폴더에서 L0 run artifact 생성까지 성공	GitHub PR comment/artifact loop 까지 성공
Test pass rate	핵심 unit/integration/e2e 테스트 80% 이상	핵심 테스트 95% 이상 및 regression suite 보유
Scope violation	forbidden path 수정 시 차단 또는 실패 보고	policy engine 이 모든 execution profile 에 적용
Documentation quality	README, PRD, test spec, demo script 존재	사용자 가이드, API docs, architecture docs 존재
Portfolio clarity	3~5 분 데모 영상으로 설명 가능	Dashboard 와 실제 PR 예시까지 제시 가능

3. Scope – MVP 부터 완성형까지

3.1 MVP 범위

- pnpm 기반 TypeScript monorepo 로 meta-harness-platform 생성
- CLI skeleton: doctor, scaffold planning, plan synthesize, plan freeze, factory bootstrap, harness run
- Target Repo 에 planning-only scaffold 생성
- build-handoff.json, acceptance-criteria.json, verification-map.json 생성
- approved Build Handoff 없이는 factory bootstrap 차단
- factory.yml, state.yml, manifest.lock, execution-profiles.yml 생성
- L0_LOCAL_WORKTREE runner 구현
- Shell Adapter 또는 Codex Adapter 1 개 구현
- patch.diff, run-result.json, summary.md artifact 생성
- 핵심 unit/integration/e2e 테스트 작성

3.2 완성형 범위

- Agent Adapter Layer: Codex, Claude, OpenHands, Shell Adapter 확장 가능 구조
- Execution Profiles: L0 worktree, L1 container, L2 kind namespace, L3 GitHub Action
- GitHub PR comment + artifact loop
- Productization Harness: UX/A11Y/Performance/Security audit
- Release Harness: compose/Caddy/GitHub Actions 기반 배포
- Maintenance Harness: dependency/security update, incident report
- Sanitized Feedback / Eval Registry / Upgrade PR loop
- React Dashboard: run history, artifact viewer, gate failure viewer, project phase viewer

3.3 범위 판단 기준

질문	MVP 판단	후속 확장 판단
현재 기능이 end-to-end loop 를 닫는가?	예: 포함	아니면 보류
없어도 데모가 가능한가?	보류	완성형에서 검토
보안 경계에 직접 영향이 있는가?	우선 포함	policy engine 과 함께 확장
포트폴리오 설명력을 높이는가?	UI/문서화는 포함	고급 자동화는 후순위

4. 시스템 아키텍처

아키텍처 원칙

1. Meta 는 프로젝트를 품지 않는다.

2. Target Repo 가 기획, 코드, 인프라, 실행 산출물의 source of truth 다.

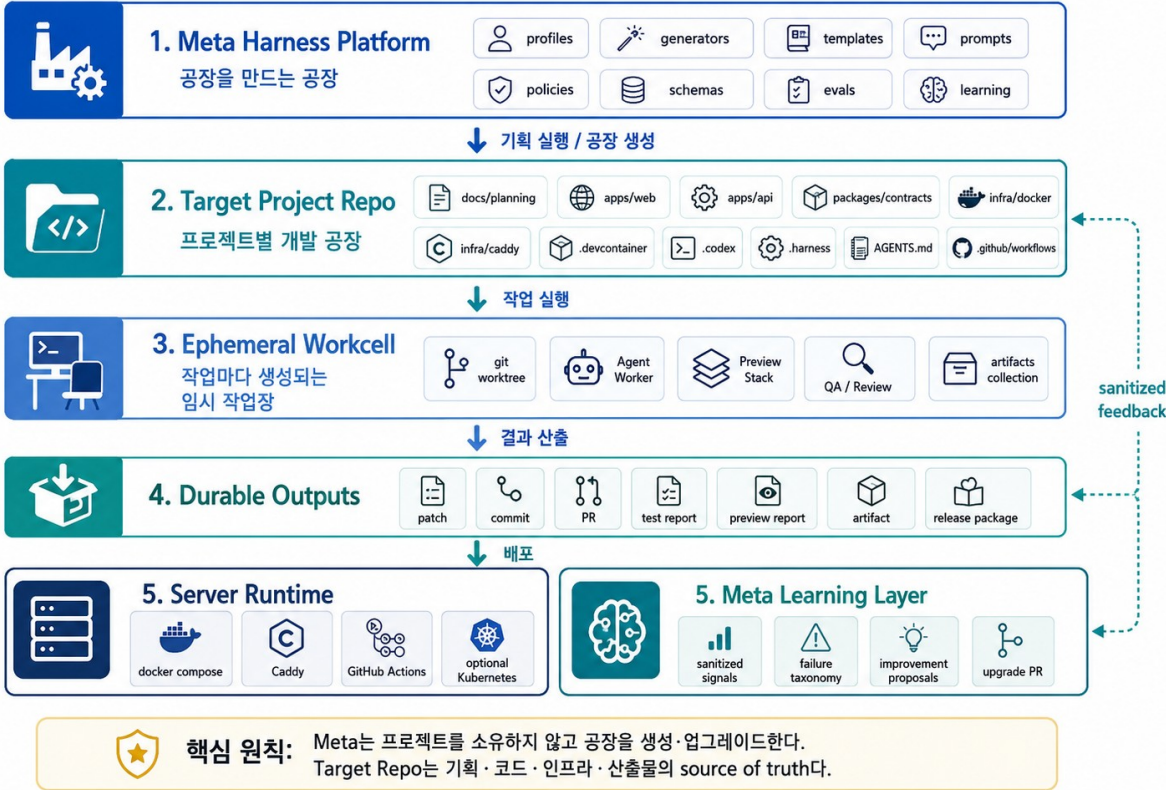
3. 모든 자동화는 state machine 과 manifest 를 통해 관리한다.

4. 모든 작업은 typed Task Packet 과 Execution Profile 을 통해 실행한다.

5. Meta Learning 은 raw data 가 아니라 sanitized signal 만 사용한다.

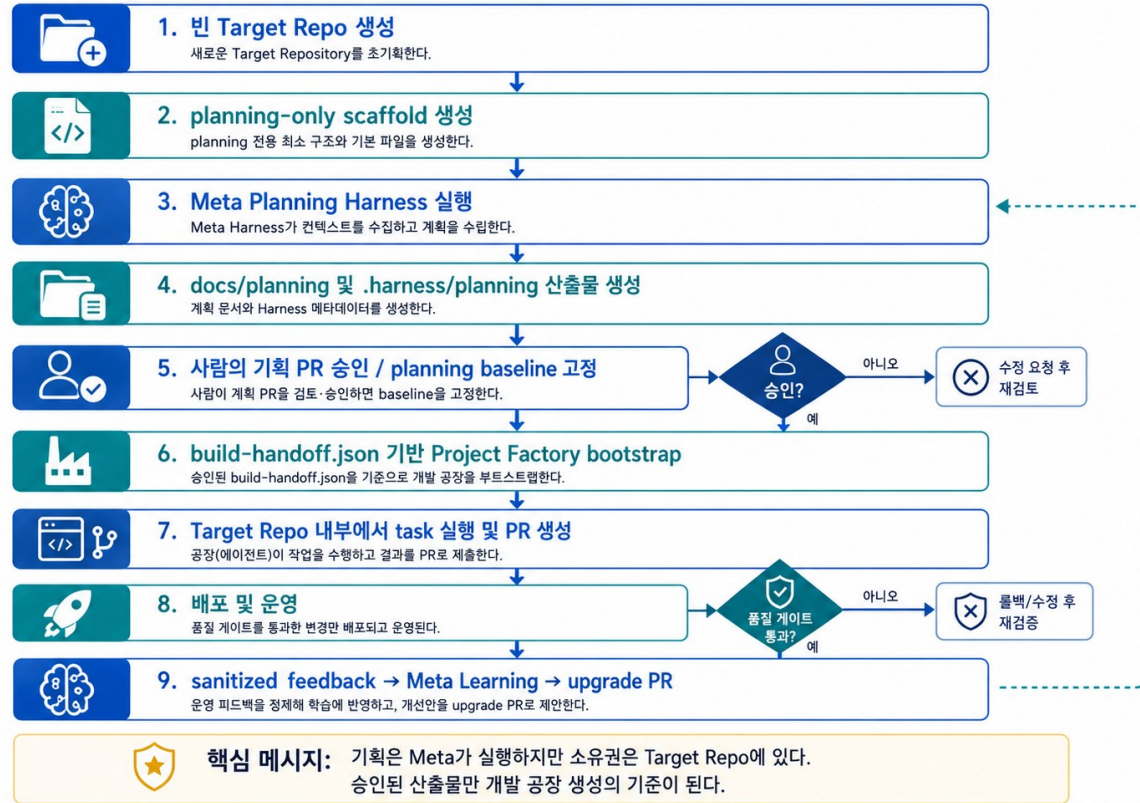
01. 전체 아키텍처 개요

Meta Harness / Target Project Repo / Workcell / Runtime / Learning Layer



02. 전체 생애주기 흐름

빈 Target Repo에서 배포·학습 루프까지



F-02. 전체 생애주기 흐름

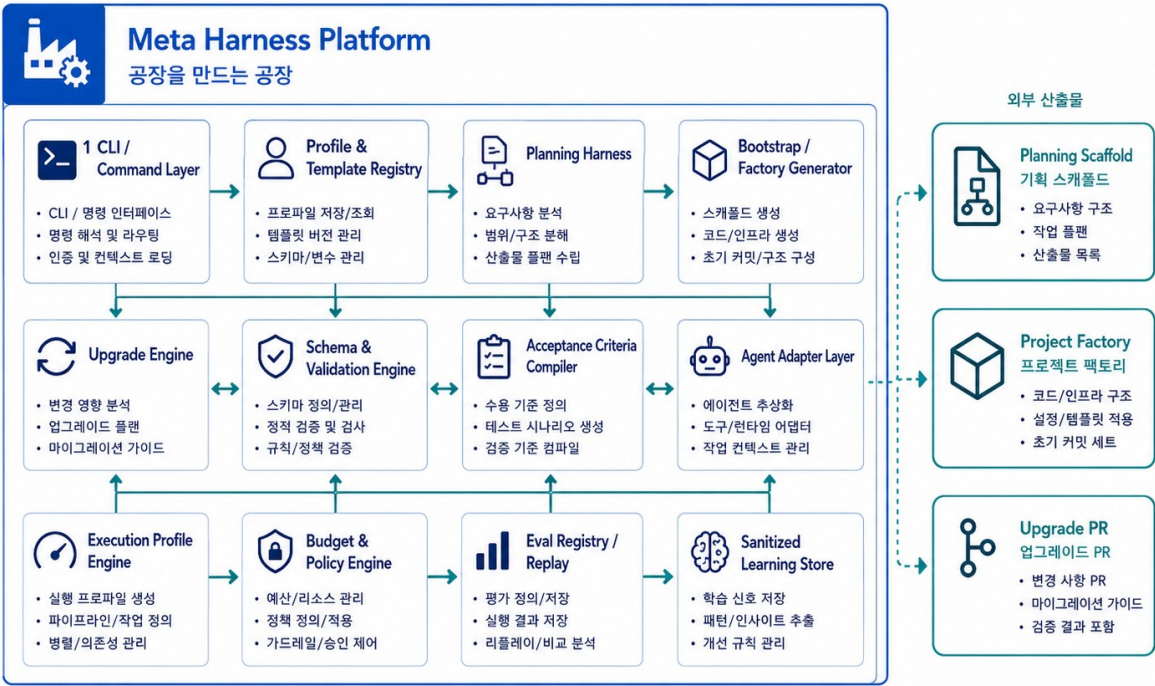
4.1 컴포넌트 구조

컴포넌트	내부 요소	구현 우선순위
Meta Harness Platform	CLI, generator, schema engine, acceptance compiler, adapter engine, eval registry	P0~P2
Target Project Repo	docs, apps, packages, infra, .harness, .codex, .github, AGENTS.md	P0~P1
Workcell	git worktree, container worker, kind namespace, GitHub Action	L0 먼저
Artifact Layer	patch.diff, run-result.json, summary.md, test-report, screenshot	P0
Learning Layer	sanitized signal, failure taxonomy, improvement proposal, upgrade PR	P2 이후

4.2 상세 아키텍처 다이어그램

03. Meta Harness Platform 상세 구조

공장을 만드는 공장의 내부 구성



Meta는 프로젝트 원문 데이터를 저장하지 않고, 범용 템플릿·정책·평가·개선 규칙을 관리한다.

F-03. Meta Harness Platform 상세 구조

04. Target Project Repo 상세 구조

프로젝트별 개발 공장의 소유 구조



핵심 원칙: Target Repo는 기획 · 코드 · 인프라 · 실행 산출물의 **source of truth**

F-04. Target Project Repo 상세 구조

05. Planning Harness 상세 흐름

아이디어를 승인 가능한 Build Handoff로 변환



핵심 원칙: 기획은 사람이 승인해야 하며, 승인 전에는 full Project Factory를 생성하지 않는다.

F-05. Planning Harness 상세 흐름

5. 기능 요구사항

ID	요구사항	설명	우선순위
FR-001	Planning scaffold 생성	빈 Target Repo 에 docs/planning, .harness/project.yml, .harness/state.yml 을 생성해야 한다.	P0
FR-002	Planning synthesis	입력 답변을 기반으로 PRD, MVP Scope, Acceptance Criteria, Build Handoff 를 생성해야 한다.	P0
FR-003	Planning freeze	사람의 승인 없이는 planning-frozen 상태로 전이되지 않아야 한다.	P0
FR-004	Factory bootstrap	approved build-handoff.json 을 기준으로 apps, infra, .harness, .codex, Makefile 을 생성해야 한다.	P0
FR-005	Manifest registration	생성된 파일은 manifest.lock 에 template version, checksum, ownership, mergeStrategy 와 함께 등록되어야 한다.	P0
FR-006	Task Packet compile	Issue/backlog item 에서 objective, scope, AC, verify commands, budgets, artifacts 를 가진 task-packet.json 을 생성해야 한다.	P0
FR-007	L0 worktree run	작업마다 git worktree 를 만들고 adapter 실행 후 artifact 를 회수해야 한다.	P0
FR-008	Agent Adapter 실행	Shell Adapter 를 기본으로, Codex Adapter 를 후속으로 실행할 수 있어야 한다.	P0/P1
FR-009	Policy enforcement	forbidden path, command policy, secret policy 를 실행 전/후 검증해야 한다.	P1
FR-010	Run artifacts	patch.diff, run-result.json, summary.md 를 표준 산출물로 생성해야 한다.	P0
FR-011	GitHub PR loop	branch/commit/PR 생성과 PR comment/artifact upload 를 지원해야 한다.	P1
FR-012	Sanitized feedback	raw data 없이 failure code, metrics, improvement signal 만 export 해야 한다.	P2
FR-013	Upgrade PR	new generator version 에 대해 Target Repo 에 upgrade PR 을 제안할 수 있어야 한다.	P2
FR-014	Dashboard	project phase, run history, artifact, gate failure 를 UI 로 볼 수 있어야 한다.	P3

5.1 비기능 요구사항

ID	속성	명세
NFR-001	재현성	동일 Task Packet 은 동일 Execution Profile 에서 재실행 가능해야 한다.
NFR-002	보안	Worker 는 기본적으로 production secret, deploy credential, Docker socket 에 접근하지 않아야 한다.
NFR-003	관찰성	모든 run 은 run-id, input hash, output hash, duration, status 를 남겨야 한다.
NFR-004	확장성	Agent Adapter 와 Execution Profile 은 pluggable 구조여야 한다.
NFR-005	사용성	주요 명령은 3~5 개 CLI 명령으로 데모 가능해야 한다.
NFR-006	문서화	각 schema, command, artifact 는 예시와 테스트 케이스를 포함해야 한다.

6. 기술 명세

6.1 Meta Harness Platform 폴더 구조

```
meta-harness-platform/  
├── apps/harness-cli/  
├── packages/  
│   ├── core/  
│   ├── generator-engine/  
│   ├── schema-engine/  
│   ├── acceptance-compiler/  
│   ├── agent-adapter-engine/  
│   ├── execution-profile-engine/  
│   ├── policy-engine/  
│   ├── worktree-runner/  
│   ├── artifact-engine/  
│   └── eval-registry/  
├── templates/  
├── profiles/  
├── schemas/  
├── examples/  
└── tests/
```

6.2 Target Project Repo 폴더 구조

04. Target Project Repo 상세 구조

프로젝트별 개발 공장의 소유 구조



핵심 원칙: Target Repo는 기획 · 코드 · 인프라 · 실행 산출물의 **source of truth**

F-04. Target Project Repo 상세 구조

```
target-project/  
├── docs/planning/  
├── apps/web/  
└── apps/api/
```

```

├── packages/shared/
├── packages/contracts/
├── infra/docker/
├── infra/caddy/
├── .harness/
│   ├── state.yml
│   ├── factory.yml
│   ├── manifest.lock
│   ├── execution-profiles.yml
│   ├── planning/
│   ├── tasks/
│   ├── runs/          # gitignore
│   └── feedback/
├── .codex/
├── .devcontainer/
├── .github/workflows/
├── AGENTS.md
├── Makefile
└── package.json

```

6.3 핵심 계약 파일

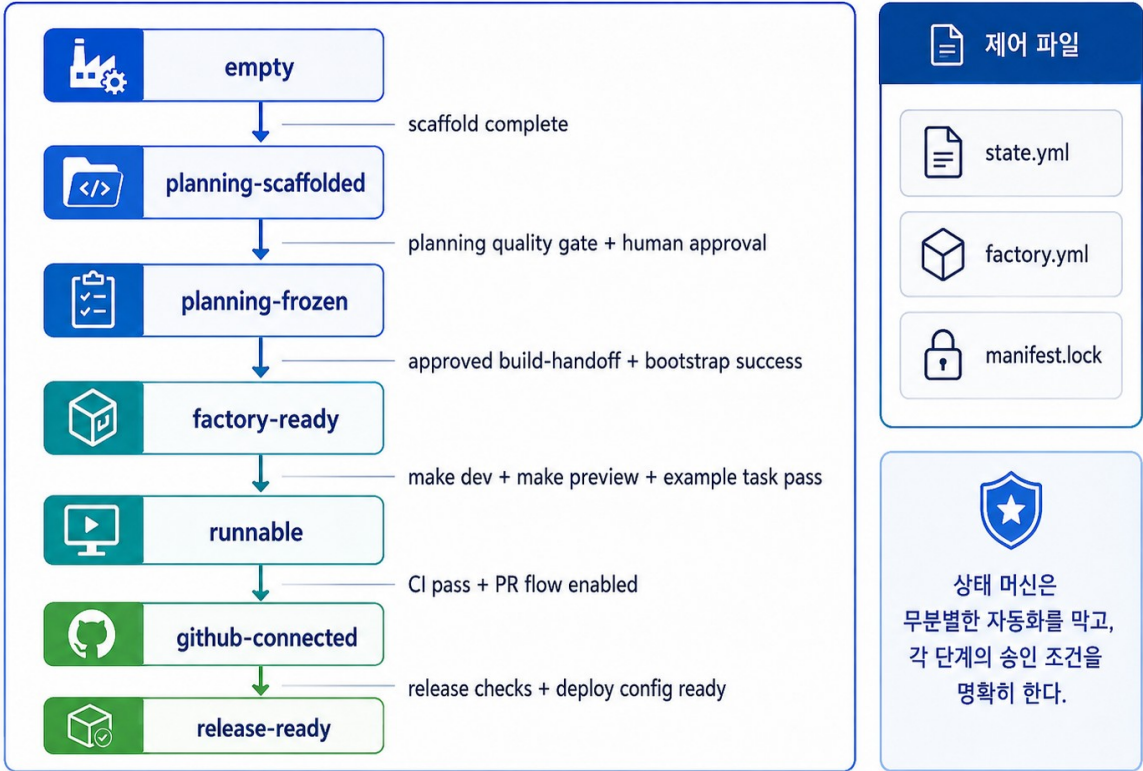
파일	역할	커밋 여부
.harness/state.yml	프로젝트 공장의 lifecycle phase 와 전이 조건 관리	O
.harness/factory.yml	공장 capability, runtime, profile, policy 정의	O
.harness/manifest.lock	생성 파일 ownership, checksum, mergeStrategy, upgrade 계약	O
.harness/execution-profiles.yml	L0/L1/L2/L3 실행 프로파일 정의	O
.harness/planning/build-handoff.json	승인된 기획 기준으로 bootstrap 입력	O
.harness/tasks/*.task.json	작업별 Task Packet	O 또는 생성
.harness/runs/*	run artifacts	X - gitignore

6.4 CLI 명령어 명세

명령	목적	산출/효과
mh doctor	환경 점검	Node, pnpm, git, optional gh/docker/codex 확인
mh scaffold planning --target <dir>	기획 스캐폴드	planning-only repo 구조 생성
mh plan synthesize --target <dir> --input <file>	기획 산출물 생성	docs/planning 및 .harness/planning 생성
mh plan compile-acceptance --target <dir>	검증 맵 생성	acceptance criteria → verification-map 생성
mh plan freeze --target <dir> --approved	기획 baseline 고정	state=planning-frozen
mh factory bootstrap --target <dir>	개발 공장 생성	approved handoff 기반 factory-ready 로 전이
mh task compile --target <dir> --source <issue>	Task Packet 생성	task-packet.json 생성
mh run --target <dir> --task <file> --adapter shell	L0 run 실행	worktree 생성, adapter 실행, artifact 회수
mh factory upgrade --target <dir> --dry-run	업그레이드 검토	manifest 기반 diff/upgrade plan 생성

06. Factory State Machine

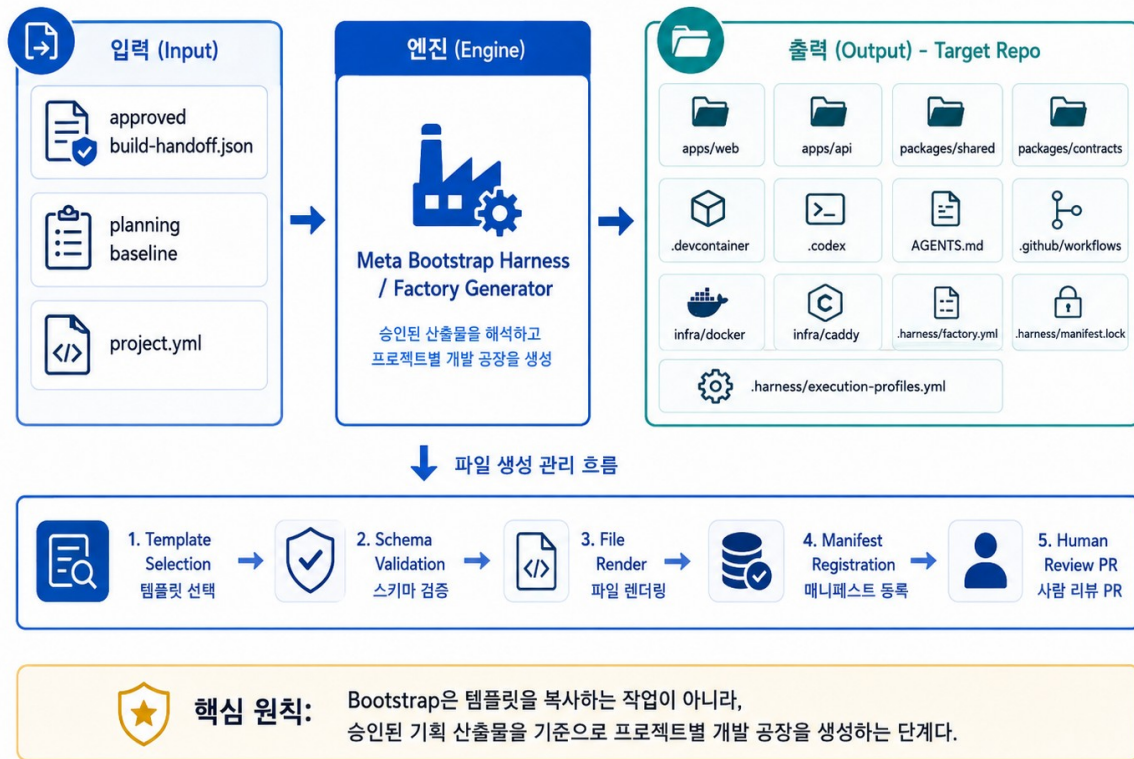
프로젝트 공장의 상태와 전이 규칙



F-06. Factory State Machine

07. Bootstrap / Factory Generation

승인된 Build Handoff를 읽고 개발 공장 생성



F-07. Bootstrap / Factory Generation

08. Workcell & Execution Profiles

동일한 Task Contract를 다양한 실행 환경에서 수행



핵심 원칙: 기본값은 L0이며, 프로젝트 성숙도에 따라 상위 profile로 확장한다.

F-08. Workcell & Execution Profiles

09. 보안 경계와 소유권 분리

Meta / Target Repo / Worker의 접근 범위 정의



F-09. 보안 경계와 소유권 분리

10. Task Packet → PR 생성 파이프라인

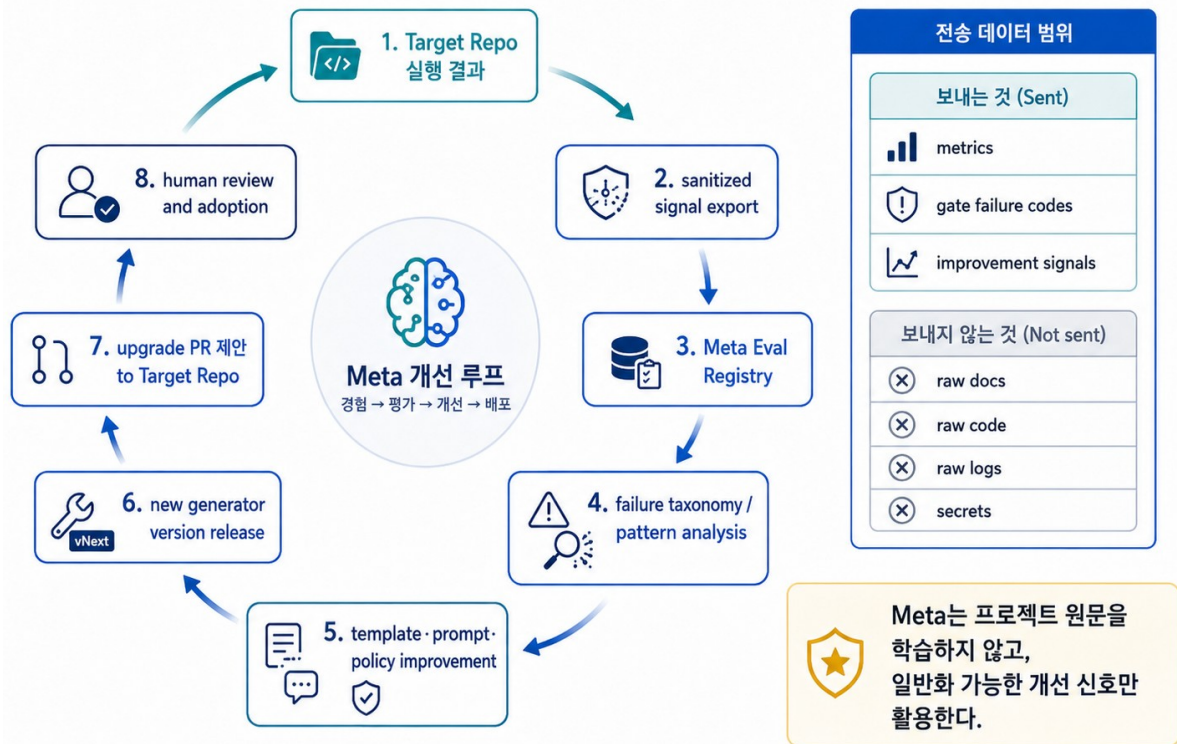
작업 계약부터 검증·산출물까지



F-10. Task Packet → PR 생성 파이프라인

11. Feedback / Eval / Upgrade Loop

Target Repo 경험을 Meta 개선으로 연결



F-11. Feedback / Eval / Upgrade Loop

7. 빈 폴더에서 시작하는 구현 절차

7.1 초기 명령 흐름

```
# 0. 빈 작업 폴더
mkdir meta-harness-workspace
cd meta-harness-workspace

# 1. Meta Harness Platform 생성
mkdir meta-harness-platform target-project
cd meta-harness-platform
pnpm init
pnpm add -D typescript tsx vitest eslint prettier
pnpm add commander zod yaml execa fs-extra simple-git

# 2. CLI skeleton 구현 후 확인
pnpm mh doctor

# 3. Target Project에 planning-only scaffold 생성
pnpm mh scaffold planning --target ../target-project --project-id demo

# 4. 기획 산출물 생성/고정
pnpm mh plan synthesize --target ../target-project --input ./examples/demo-answers.yml
pnpm mh plan compile-acceptance --target ../target-project
pnpm mh plan freeze --target ../target-project --approved

# 5. Project Factory bootstrap
pnpm mh factory bootstrap --target ../target-project

# 6. L0 worktree runner 실행
cd ../target-project
make harness-run TASK=./harness/tasks/example.task.json ADAPTER=shell
```

7.2 Sprint 단위 구현 순서

단계	구현 주제	추가하는 것	기대효과
Sprint 0	Core schema + monorepo skeleton	state/factory/manifest/task/run-result schema, CLI skeleton	명세 중심 개발로 이후 구현 흔들림 감소
Sprint 1	Planning-only scaffold	docs/planning, .harness/project.yml, state.yml	기획 승인 전 개발 공장 생성을 방지
Sprint 2	Planning Harness v1	PRD, MVP Scope, AC, Build Handoff 생성	아이디어를 구현 가능한 산출물로 변환
Sprint 3	Acceptance Compiler	verification-map, acceptance-tests.generated.json	기획과 테스트를 연결
Sprint 4	Factory Bootstrap	Target Repo apps/infra/.harness/.codex 생성	승인된 기획 기준으로 개발 공장 생성
Sprint 5	L0 Worktree Runner	git worktree, shell adapter, artifacts	작업 실행 루프를 닫음
Sprint 6	Security Boundary	secret/network/command/forbidden path policy	AI 작업의 위험 범위 제한
Sprint 7	GitHub Minimal Loop	branch/commit/PR/comment/artifact	실제 협업 흐름과 연결
Sprint 8	Eval + Feedback	sanitized signal, eval registry	일반화된 개선 루프 시작
Sprint 9+	Container/kind/Dashboard/Productization	고급 실행 profile 과 UI	완성형 하네스로 확장

8. 단계별 추가 항목과 기대효과

Phase 1 - Core Schema / CLI Skeleton

구분	내용
추가하는 것	apps/harness-cli schemas/*.schema.json packages/core mh doctor
기대효과	구현 전 계약을 고정해 기능 확장 시 혼선을 줄인다. 테스트와 문서가 schema 기준으로 정렬된다.
완료 기준	pnpm build/pass pnpm test/pass mh doctor 출력

Phase 2 - Planning-only Target Scaffold

구분	내용
추가하는 것	README.md docs/planning/ .harness/state.yml .harness/project.yml .github/pull_request_template.md
기대효과	Target Repo 가 기획 산출물의 source of truth 가 된다. 승인 전 apps/infra/.codex 생성을 막는다.
완료 기준	state.phase=planning-scaffolded apps/infra/.codex/가 없어야 함

Phase 3 - Planning Harness

구분	내용
추가하는 것	03_PRD.md 05_MVP_SCOPE.md 11_ACCEPTANCE_CRITERIA.md 14_BUILD_HANDOFF.md build-handoff.json
기대효과	아이디어가 구현 가능한 계획으로 정리된다. 기획 문서와 agent 용 JSON 산출물이 동시에 생긴다.
완료 기준	필수 필드 누락 없음 quality gate pass

Phase 4 - Factory Bootstrap

구분	내용
추가하는 것	apps/web packages/shared infra/docker .codex AGENTS.md factory.yml manifest.lock
기대효과	승인된 기획 산출물을 기준으로 프로젝트별 개발 공장이 생성된다. 생성 파일 ownership 과 upgrade 전략이 기록된다.
완료 기준	approved handoff 없으면 실패 manifest.lock 에 생성 파일 등록

Phase 5 - L0 Worktree Runner

구분	내용
----	----

구분	내용
추가하는 것	Task Packet compiler git worktree runner Shell/Codex Adapter run artifacts
기대효과	실제 작업 실행 루프가 닫힌다. 작업 결과가 patch/report 형태로 남아 포트폴리오 데모가 가능해진다.
완료 기준	patch.diff 생성 run-result.json 생성 cleanup 성공

Phase 6 – Security & Policy

구분	내용
추가하는 것	forbidden-scope.yml command-policy.yml secret-policy.yml runtime-policy.yml
기대효과	agent 가 수정 가능한 범위와 금지 범위가 명확해진다. 자동화보다 통제 가능한 실행 경계가 우선된다.
완료 기준	forbidden path 수정 차단 secret 파일 접근 실패

Phase 7 – GitHub Minimal Loop

구분	내용
추가하는 것	branch/commit/PR command workflow summary PR comment artifact upload
기대효과	로컬 도구를 실제 협업 워크플로우와 연결한다. 면접/포트폴리오에서 실무형 데모가 가능해진다.
완료 기준	PR 생성 성공 artifact 링크 제공

Phase 8 – Feedback / Eval / Upgrade

구분	내용
추가하는 것	sanitized signal schema eval registry failure taxonomy upgrade PR generator
기대효과	Meta 가 프로젝트 원문 없이도 일반화된 실패 패턴을 학습한다. factory upgrade 를 PR 로 제안할 수 있다.
완료 기준	raw docs/code/logs 미포함 upgrade dry-run diff 생성

9. 테스트 명세서

9.1 테스트 전략

테스트 유형	대상	목표
Unit	schema validators, compiler, generator utilities	작은 함수와 계약 검증
Integration	scaffold → plan → freeze → bootstrap	단계 간 파일 생성과 state 전이 검증
E2E	빈 폴더에서 run artifact 생성까지	MVP 데모 경로 보장
Security	forbidden path, secret, command policy	agent 작업 경계 검증
Upgrade	manifest.lock, checksum, managed-blocks	생성 파일 drift와 upgrade PR 품질 검증
Regression	golden target repo, golden task packet	새 generator 버전이 기존 기능을 깨지 않는지 검증

9.2 핵심 테스트 케이스

ID	시나리오	절차	기대 결과
T-001	빈 폴더 doctor	mh doctor 실행	필수 도구 상태와 경고를 출력한다.
T-010	planning scaffold 생성	빈 target-project 에 scaffold planning 실행	docs/planning, .harness/state.yml 생성, apps/는 생성되지 않음
T-011	중복 scaffold 보호	동일 target 에 scaffold 재실행	기존 파일을 덮어쓰기 전 경고 또는 idempotent 처리
T-020	planning synthesize	demo-answers.yml 입력	PRD, MVP Scope, Acceptance Criteria, Build Handoff 생성
T-030	acceptance compiler	acceptance-criteria.json 입력	verification-map.json 생성
T-040	승인 전 bootstrap 금지	state=planning-scaffolded 에서 factory bootstrap	FACTORY_BOOTSTRAP_REQUIRES_PLANNING_FROZEN 실패
T-041	승인 후 bootstrap 성공	plan freeze --approved 후 bootstrap	apps, infra, .harness/factory.yml, manifest.lock 생성
T-050	manifest 등록 검증	bootstrap 이후 manifest.lock 확인	모든 generated file 이 checksum 과 ownership 을 가진다.
T-060	Task Packet compile	backlog item 에서 task 생성	objective, editableScope, forbiddenScope, AC, verifyCommands 포함
T-070	L0 worktree run	make harness-run ADAPTER=shell	worktree 생성, adapter 실행, artifact 회수, cleanup
T-071	run artifact 생성	run 완료 후 .harness/runs/run-id 확인	patch.diff, run-result.json, summary.md 존재
T-080	forbidden path 차단	task 가 .env 수정 시도	run 실패 및 POLICY_FORBIDDEN_PATH 기록
T-081	금지 명령 차단	rm -rf 또는 docker login 실행 시도	command policy 가 차단
T-090	upgrade dry-run	새 template version 으로 upgrade dry-run	diff 와 upgrade-plan.md 생성, 자동 적용 안 함
T-100	sanitized signal export	실패 run 을 feedback export	raw docs/code/logs/secrets 없이 metrics/reasonCode 만 포함
T-110	GitHub PR loop	gh CLI 가 활성화된 repo 에서 PR 생성	branch/commit/PR 생성 및 summary 첨부

9.3 인수 테스트 시나리오

Scenario: MVP end-to-end run
Given an empty workspace
When the user creates meta-harness-platform and target-project
And runs scaffold planning, plan synthesize, plan freeze, factory bootstrap
And executes make harness-run TASK=.harness/tasks/example.task.json ADAPTER=shell
Then the Target Repo should contain a factory-ready state
And the run directory should contain patch.diff, run-result.json, summary.md
And the temporary worktree should be cleaned up
And no forbidden path should be modified

10. 완료 기준과 인수 기준

10.1 MVP Definition of Done

- README 의 명령 흐름을 복사해서 빈 폴더에서 그대로 실행할 수 있다.
- planning-only scaffold 생성과 full factory bootstrap 이 상태 머신으로 분리되어 있다.
- approved Build Handoff 없이는 bootstrap 이 실패한다.
- factory.yml, state.yml, manifest.lock, execution-profiles.yml 이 생성된다.
- Task Packet 에서 L0 worktree run 을 실행할 수 있다.
- patch.diff, run-result.json, summary.md 가 생성된다.
- 최소 unit/integration/e2e/security 테스트가 통과한다.
- 3~5 분 데모 영상으로 문제-해결-실행-결과를 설명할 수 있다.

10.2 완성형 Definition of Done

- Shell/Codex/Claude 등 Agent Adapter 를 추가할 수 있는 구조가 안정화되어 있다.
- L1 container worker 와 L2 kind namespace profile 이 선택적으로 동작한다.
- GitHub PR comment + artifact loop 가 동작한다.
- Productization audit 와 release readiness report 를 생성할 수 있다.
- sanitized signal 을 Meta eval registry 로 export 할 수 있다.
- factory upgrade dry-run 과 upgrade PR 생성이 동작한다.
- Dashboard 에서 project state, run history, artifact, gate failure 를 볼 수 있다.

10.3 MVP 실행 성공 기준

```
cd meta-harness-platform
pnpm install
pnpm build
pnpm test
pnpm mh doctor
pnpm mh scaffold planning --target ../target-project --project-id demo
pnpm mh plan synthesize --target ../target-project --input ./examples/demo-answers.yml
pnpm mh plan compile-acceptance --target ../target-project
pnpm mh plan freeze --target ../target-project --approved
pnpm mh factory bootstrap --target ../target-project
cd ../target-project
make harness-run TASK=../harness/tasks/example.task.json ADAPTER=shell
```

11. 리스크 및 대응

리스크	영향	대응
범위 과다	완성 전 피로도 증가, 포트폴리오화 실패	MVP 는 Shell Adapter + L0 worktree 만으로 제한한다.
AI agent 종속성	Codex/Claude 정책 변경 시 프로젝트 흔들림	Agent Adapter Layer 로 추상화하고 Shell Adapter 를 기본 회귀 테스트로 둔다.
템플릿 drift	생성 파일과 수동 수정 파일 충돌	manifest.lock, ownership, mergeStrategy, upgrade PR 로 관리한다.
보안 경계 붕괴	secret 노출, production 파일 수정 위험	setup/worker phase 분리, forbidden scope, secret policy, command policy 도입
Kubernetes 조기 도입	디버깅 비용 증가	L0 worktree-first, kind 는 후순위 profile 로 둔다.
문서만 있고 작동하지 않음	커리어 효과 낮음	E2E demo 와 run artifact 를 최우선 완성 기준으로 둔다.
면접관 이해 난이도	복잡한 시스템으로 보일 수 있음	3 분 데모 스크립트와 Before/After 문제 정의를 준비한다.

12. 포트폴리오 활용 전략

커리어 포지셔닝

이 프로젝트의 커리어 포지션은 "AI Researcher"가 아니라 "AI Workflow 를 이해하는 프론트엔드/풀스택 DevTools 개발자"다.

12.1 GitHub README 구성

- 문제 정의: AI coding agent 의 기획/범위/검증/보안 경계 부족
- 해결 구조: Meta Harness → Target Repo → Workcell → Artifacts → Feedback
- 빠른 시작: 빈 폴더에서 MVP 실행까지 명령어
- 다이어그램: 전체 아키텍처, lifecycle, task pipeline
- 데모 GIF/영상: scaffold, bootstrap, run artifact 생성
- 테스트 결과: unit/integration/e2e/security test pass
- 로드맵: MVP, GitHub loop, Dashboard, container/kind, eval/upgrade

12.2 이력서 bullet 예시

- TypeScript/pnpm 기반 CLI 로 빈 Target Repo 에 planning scaffold 와 development factory 를 자동 생성하는 AI-assisted SDLC prototype 구현
- build-handoff.json, task-packet.json, manifest.lock, execution-profiles.yml 등 schema-driven contract 설계
- git worktree 기반 ephemeral runner 를 구현해 작업별 patch.diff, run-result.json, summary.md artifact 생성
- forbidden path, secret boundary, execution profile 을 통해 agent 작업 범위와 보안 정책을 명시적으로 제한
- React dashboard 를 추가해 project phase, run history, gate failure, patch summary 를 시각화할 수 있도록 설계

12.3 데모 스크립트

15. 빈 폴더에서 meta-harness-platform 과 target-project 를 만든다.
16. planning-only scaffold 를 생성하고 아직 apps/infra/.codex 가 없음을 보여준다.
17. 기획 산출물과 build-handoff.json 을 생성한다.
18. human approval 후 factory bootstrap 을 실행한다.
19. L0 worktree runner 로 example task 를 실행한다.
20. patch.diff, run-result.json, summary.md 를 보여준다.
21. forbidden path 를 수정하려는 실패 케이스를 보여준다.

Appendix A. 계약 파일 예시

A.1 state.yml

```
schemaVersion: 1
projectState:
  phase: factory-ready
current:
  planningBaselineHash: sha256:...
  buildHandoffHash: sha256:...
  factoryManifestHash: sha256:...
allowedTransitions:
- from: planning-scaffolded
  to: planning-frozen
  requires:
    - planningQualityGatePassed
    - humanApproval
- from: planning-frozen
  to: factory-ready
  requires:
    - approvedBuildHandoff
    - bootstrapSuccess
```

A.2 manifest.lock

```
schemaVersion: 1
generator:
  name: meta-harness-platform
  version: 0.4.0
source:
  buildHandoff: .harness/planning/build-handoff.json
  approvedCommit: <git-sha>
files:
- path: AGENTS.md
  ownership: shared
  template: codex/AGENTS.md.hbs
  checksum: sha256:...
  mergeStrategy: managed-blocks
  conflictPolicy: require-human-review
- path: .github/workflows/ci.yml
  ownership: harness
  template: github-actions/ci.yml.hbs
  checksum: sha256:...
  mergeStrategy: replace-if-unchanged
  conflictPolicy: propose-only
```

A.3 task-packet.json

```
{
  "schemaVersion": "1.0.0",
  "taskId": "ISSUE-003",
  "taskType": "frontend-ui",
  "objective": "Implement dashboard vertical slice.",
  "editableScope": ["apps/web/src/**", "tests/**"],
  "forbiddenScope": [".env*", "infra/**/production/**"],
  "acceptanceCriteria": [
    {
      "id": "AC-003-01",
      "text": "Dashboard renders loading, empty, error, and success states.",
      "verification": { "type": "command", "command": "pnpm test -- dashboard" }
    }
  ],
  "commands": {
    "verify": ["pnpm lint", "pnpm typecheck", "pnpm test"]
  },
  "artifacts": ["patch.diff", "run-result.json", "summary.md"],
  "budgets": { "maxRuntimeMinutes": 20, "maxPatchLines": 800 }
}
```

A.4 sanitized-signal.json

```
{
  "schemaVersion": "1.0.0",
  "source": {
```



```
"factoryProfile": "web-app",
"generatorVersion": "0.4.0",
"executionProfile": "L0_LOCAL_WORKTREE"
},
"task": {
  "taskType": "frontend-ui",
  "complexityBucket": "small"
},
"result": {
  "status": "failed",
  "failureCategory": "missing-responsive-rules",
  "gateFailures": [
    { "gate": "planning-quality-gate", "reasonCode": "missing-responsive-rules" }
  ]
},
"privacy": {
  "rawCodeIncluded": false,
  "rawDocsIncluded": false,
  "rawLogsIncluded": false,
  "secretsIncluded": false
}
}
```

Appendix B. 다이어그램 카탈로그

번호	이름	용도
F-01	전체 아키텍처 개요	문서/발표/README 에서 아키텍처 설명용
F-02	전체 생애주기 흐름	문서/발표/README 에서 아키텍처 설명용
F-03	Meta Harness Platform 상세 구조	문서/발표/README 에서 아키텍처 설명용
F-04	Target Project Repo 상세 구조	문서/발표/README 에서 아키텍처 설명용
F-05	Planning Harness 상세 흐름	문서/발표/README 에서 아키텍처 설명용
F-06	Factory State Machine	문서/발표/README 에서 아키텍처 설명용
F-07	Bootstrap / Factory Generation	문서/발표/README 에서 아키텍처 설명용
F-08	Workcell & Execution Profiles	문서/발표/README 에서 아키텍처 설명용
F-09	보안 경계와 소유권 분리	문서/발표/README 에서 아키텍처 설명용
F-10	Task Packet → PR 생성 파이프라인	문서/발표/README 에서 아키텍처 설명용
F-11	Feedback / Eval / Upgrade Loop	문서/발표/README 에서 아키텍처 설명용

사용 권장

README 에는 F-01, F-02, F-10 을 우선 배치한다.
기술 문서에는 F-03~F-09 를 배치한다.
포트폴리오 발표에는 F-01 → F-02 → F-10 → F-11 순서가 가장 이해가 쉽다.